

基於基因演算法之任意角度且考慮屏障之裸晶間繞線演算法

Die-to-Die Routing at Arbitrary Angles with Shielding Awareness Based on a Genetic Algorithm

1st Wu Zong-Qiao

Department of Computer Science

National Tsing Hua University

Hsinchu, Taiwan

yaudksodop@gmail.com

2nd Tsai Wei-Chen

Department of Electrical Engineering

National Tsing Hua University

Hsinchu, Taiwan

thenxt2.0@gmail.com

Abstract—隨著晶片的封裝技術的進步，台積電提出新式的集成扇出型封裝(Integrated fan-out, InFO)技術並應用於先進製程中，此封裝技術會透過重佈線層(Redistribution layer, RDL)將多顆裸晶以併行方式進行連接，最終集成封裝成一顆晶片。然而裸晶間繞線問題面臨的是與傳統繞線問題截然不同的挑戰，在InFO技術下的裸晶間繞線允許使用任意角度的金屬線進行連接，但因製程技術的原因，連接點與金屬線還有不同層連接點在繞線上都需要加入特殊的設計規則以保證製程的成功率，且為了保證信號完整性(Signal integrity)，每一條的信號線都需要透過接地線進行隔離(Ground shielding)以防止不同信號線間的雜訊干擾，這些特殊限制導致傳統繞線演算法難以應用於裸晶間繞線上，目前已有的演算法僅能應用於具有特定連接關係的關係上，因此在本論文中，我們提出以基因演算法實現的裸晶間繞線演算法，演算法會先基於德勞內三角形建立繞線模型，之後根據當層要繞的點將三角形模型轉換為雙通道模型，並使用基因演算法進行全局繞線，最終根據繞線規劃在每一個三角形內進行細部繞線，透過此演算法不只可以在盡可能減少繞線層數與長度的情況下達成隔離全部信號線的目標，同時也能應用於不規律的連接關係上。

Index Terms—Die-to-die Routing, Channel Routing, Shielding, Genetic Algorithm

I. INTRODUCTION

在現今的晶片技術下，晶片的複雜度大幅提升，因此開始發展出不同的晶片架構以增加晶片的效能表現，而目前台積電透過了集成扇出型封裝技術(Integrated fan-out, InFO)將多個裸晶一起封裝在一個晶片中，並透過重連接層進行併排連接，將整個系統直接連接並封裝在一起，相較於傳統印刷電路板(Printed circuit board, PCB)的連接方式，InFO大幅減少了裸晶間的距離並有效提升整個系統的效能。如Fig. 1就是InFO封裝技術的剖面圖，在最上層會有多顆裸晶透過 μ Bump與重佈線層進行連接，而中間有多層重佈線層，每層重佈線層會透過導通孔(後稱Via)與金屬線分別進行垂直與水平方向的連接，而裸晶間就可以透過多層重佈線層進行溝通，並透過最底層的重佈線層把晶片信號通過基板再以球柵陣列封裝(Ball grid array, BGA)將信號傳輸至封裝外。

然而InFO技術所需要的重佈線層佈局相對傳統晶片佈局有著非常大的不同，重佈線層的繞線規則更類似於PCB電路，重佈線層繞線允許任意夾角大於90度的繞線，且重

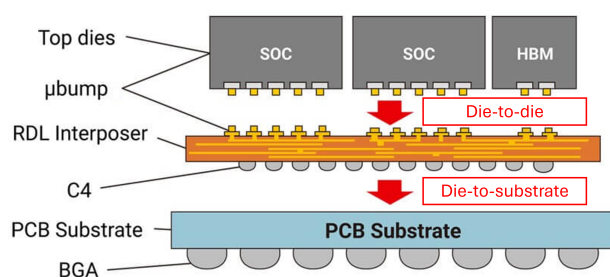


Fig. 1. InFO封裝技術剖面圖。

佈線層間的垂直連接是需要採用Offset via架構(意即上一層Via與這一層Via之間要間距一定距離)，導致垂直連接的成本高於晶片繞線的Stacked via架構，如Fig. 2，而Offset vias會減少點與點之間的繞線資源並增加重佈線層繞線的複雜度(向不同方向位移會影響繞線結果)，因此在實作上會更傾向在當層選擇繞線的點後，在同一層對這些點進行無交錯繞線(無交錯就不需要換層繞線)，以減少垂直繞線帶來的複雜度。而為了增加重佈線層的繞線良率，在接點(後稱Bump)與金屬線的連接處會需要引入淚滴型金屬線，如Fig. 3，因此在繞線上需要考量額外的幾何形狀，以保證淚滴型金屬線可以順利與對應金屬線連接，且不與其它金屬線出現接觸情況。此外，為了維持裸晶間的信號完整性，因此每一條信號線之間都需要由接地線進行隔離(Ground shielding)，如Fig. 4，以減少電磁效應產生的雜訊來穩定信號，因此在繞每一條信號線的過程裡，同時也要考慮附近是否有足夠的位置去產生接地線以包覆信號線。

因為上述的特殊規則，導致過去傳統佈局主流的繞線演算法難以的應用於重繞線層上，因此目前裸晶間重佈線層的佈局仍然依靠手拉方式來實現，然而，隨著晶片製程進步，裸晶的I/O接口也變得更加密集，重佈線層的連接關係變得更加複雜，導致佈局逐漸難以以手拉的方式完成，所以重佈線層的繞線自動化變得越來越重要，而本論文

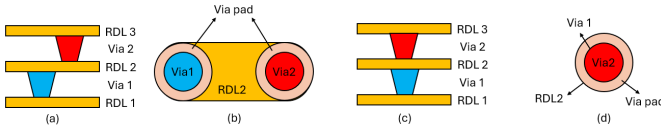


Fig. 2. (a)Offset via剖面圖(b)Offset via俯視圖(c)Stacked via剖面圖(d)Stacked via俯視圖。

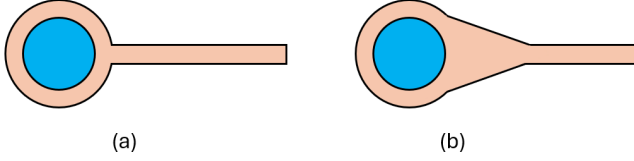


Fig. 3. (a)無淚滴型金屬線的連接(b)加入淚滴型金屬線的連接。

為了解決此問題提出了一套演算法來實現重佈線層自動繞線，以期減少InFO晶片的設計難度與開發週期。

在本論文中，我們的貢獻在於：

- 設計了專門處理重佈線層佈局自動化的演算法，在繞線成功率100%的情況下滿足重佈線層的特殊規則並盡可能減少了繞線長度。
- 提出透過基因演算法進行全局繞線的方法，並建立了此問題所需的表示法、初始化方法、交配/變異操作與適應性的計算方式。
- 以實驗結果展示在本問題下中，我們基於基因演算法所實現的全局繞線表現會比傳統基於使用最短路徑演算法搭配Rip-up & Reroute有同等的能力，但在成本函數的使用上有更高的自由度。

而後續的文章將分為5個章節，分別介紹相關文獻、問題描述、演算法、實驗結果與結論等部分。

II. RELATED WORK

[2]是第一篇提出將基因演算法應用於VLSI通道繞線問題的論文，他們透過三維的整數串表示金屬線在三維空間下的繞線結果，並透過隨機延伸的方式產生初始電路，在交配操作上則是選擇X軸上的一點 x_c ，將父母染色體所有經過 x_c 的水平線段全部移除後，再把父母的左右部分(以 x_c 為分割點)進行合併，最後對未相接的點進行隨機延伸進行連接來生成子代，在突變操作上，則採用刪去特定線段、列數或區塊的線路，再對被移除的點進行隨機延

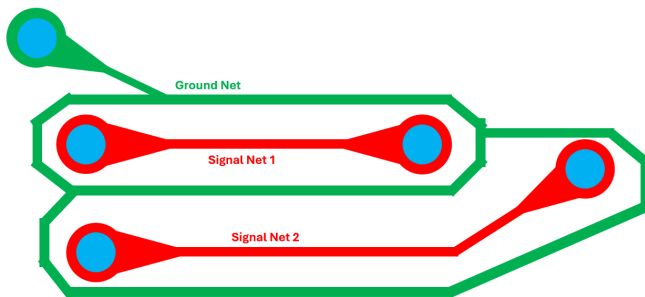


Fig. 4. 接地線隔離信號線表示圖(紅色為信號線，綠色為接地線)。

伸並連接的方式進行突變，在適應性的計算上，會先比較使用的通道數(Y軸所需的最小長度)，若通道數相同則考慮線長與Vias的數量，最後以上述操作搭配些選擇策略以完成當時的通道繞線問題，而實驗結果證明其方法在通道繞線下是可以成功收斂並找到優質的繞線結果。

[3]提出了基於德勞尼三角形進行電路分割並考慮障礙物的重佈線層繞線，它們根據電路上的Bumps與障礙物的邊界點以德勞尼三角化的方式建立了繞線圖，並基於A*-search的方式搭配Chord-based model來進行無交錯的繞線規劃，最後透過Access point determination的方法來決定真正的繞線位置，以減少繞線結果出現不必要的彎曲。

[1]則是基於 [3]的方法，並額外考慮了淚滴型凸出、Offset via與Ground shielding等問題所實現的重佈線層繞線，它們先將只有Bump的電路進行Delaunay triangulation，接著會先對Offset via所延伸的位置進行規劃，使每一列可用的通道數平均，接著根據他們所提出的S-route方法，以Bump的位置決定繞線順序後，再基於 [3]所提出的繞線方法進行繞線，雖然其方法與我們的問題相似且實驗結果也顯示它確實可以在滿足繞線規則與100%接地線隔離的情況下減少線長與層數，然而它們的S-route順序僅能應用於特定的連接關係上，因此缺乏了一些通用性。

III. PROBLEM FORMULATION

在本章節中會逐步定義要處理的問題、限制以及術語，並於下一章說明完整的演算法。

在本問題裡，每顆裸晶上有Signal bump(信號點)、Vdd bump(電源點)、Vss bump(接地點)三種接點，分別由Signal net(信號線)、Vdd net(電源線)與Vss net(接地線)進行連接，而這些Bumps會分佈在Die1(左區裸晶)與Die2(右區裸晶)上，繞線的目標是把每個的Vdd/Vss bump與其他Vdd/Vss bumps連在一起，而Signal bump要與其對應的Signal bump連在一起，且每一條Signal net都要由Vss net包覆住以實現Grounding shielding，所有Nets都允許夾角大於90度的繞線，但不能出現繞路走法(意即每一個Bump的繞線路徑只能在相鄰上下兩列之間)，且繞線結果在幾何上要滿足設計規則檢查(Design rule check, DRC)的限制，此設計規則的建模是參考 [1]的設計，如Fig. 5，而電路要滿足設計規則，使其：

- 每一條金屬線的寬度都超過最小線寬 L_w ，且任意兩條不同的金屬線(或金屬線與Via pad)之間至少距離 L_b 單位且不能出現重疊
- 每一個Via pad與其Offset via pad之間至少距離 V_s ，而Via與Via pad的半徑分別為 V_o 與 V_p
- 每條從Via延伸出去的金屬線都會有淚滴型突出，並由Via中心延伸 T_d 長度，並以 V_p 至 L_w 的寬度隨延伸距離線性縮減

繞線目標是在滿足以上情況下盡可能減少Signal net的總線長。

在Bumps的分佈上，每一列的Bumps以交錯的方式進行排列，且所有相鄰Bumps間的距離為 r ，而Signal bumps只會集中在Die的中間部分(意即同Die中的任意Signal bumps之間不會有任何Vdd/Vss bumps，但可以出現在周圍，不論左右上下)，而每個在Die1的Signal bump在Die2的同一列都會有唯一一組對應Signal bumps，後稱這一組Signal bumps為Signal pair，如Fig. 6為Die上的Bumps分布圖。

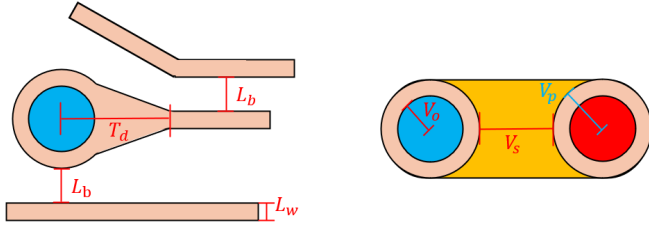


Fig. 5. 設計規則一覽。

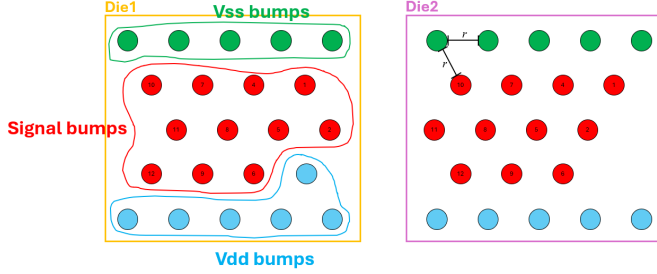


Fig. 6. Die上的Bumps分佈表示圖，紅框為Signal bumps、綠框為Vss bumps、藍框為Vdd bumps，每個Signal bump在Die1與Die2的同一列都有一組配對(相同數字的為一組)，Signal bumps只會集中在中間，而每個相鄰Bumps間的距離固定為 r 。

IV. ALGORITHM

本章節會先介紹我們演算法的整體流程，接著再介紹主要步驟在實現上的細節，我們的演算法流程如Algorithm 1，會以Bumps的資訊與設計規則作為輸入，最後回傳繞線後的設計結果，演算法的流程會迭代直到所有Signal pairs都被繞完為止，內部流程會先選擇當層要繞的Signal pairs，並將沒有要繞線的Signal bumps作為Via並水平向左產生Offset via，並於之後在更上層的電路進行繞線，接著以Delaunay triangulation來建立繞線圖，之後Global route以基於基因演算法的方式進行繞線規劃，最後Detailed route以基於貪婪演算法的方法決定實際線路的位置，如果Global route或Detailed route無法滿足限制的話就會重選本層要繞的Signal pairs，並重新繞線，直到繞出來後再繼續繞下一層。

A. Delaunay Triangulation

德勞尼三角化是一個常用於電腦視覺進行分割的方法，在我們的演算法中，會透過三角化的方式將整個電路從極高自由度的任意繞線，分割成有限的三角格格區塊並建立連接形成繞線圖(Routing graph)，再進行下一步的繞線，以減少整個問題的複雜度。首先將Die1與Die2的Bumps根據其邊界與Bumps間的距離 r ，以Padding bumps對沒有任何Bump的位置進行填充，接著透過德勞尼三角化將整個電路根據原本Bumps與Padding bumps進行三角化，得到Die1與Die2的分割圖。接著先定義Tile node為三角格的點，其中Bump-to-bump edge為Bump與相鄰Bump之間的連接，Bump-to-tile edge為Bump與相鄰Tile node之間的連接，Tile-to-tile edge為Tile node與相鄰Tile node之間的連接，而每個Tile-to-tile edge之間有個容量(Capacity)，代表了這兩個Tile nodes之間最多可以經過的繞線數量。再得到Die1與Die2的繞線圖後，我們會將兩個繞線圖對應的列

Algorithm 1 Die-to-Die Routing Algorithm

```

1: Input: Vss Bumps ( $V_{ss}$ ), Vdd Bumps ( $V_{dd}$ ), Signal Bumps ( $S$ ), Design Rule ( $Rule$ ), Genetic Algorithm Parameters ( $Parameters$ )
2: Output: Routing Result ( $Result$ )
3:  $l \leftarrow 1$ 
4:  $Result \leftarrow \{\}$ 
5: while  $S \neq \emptyset$  do
6:    $S_l \leftarrow SelectRoutingPairs(S)$ 
7:    $S'_l \leftarrow S - S_l$ 
8:    $S'_l \leftarrow S'_l \cup CreateOffsetVias(S'_l)$ 
9:    $graph_l \leftarrow DelaunayTriangulation(S_l, S'_l, V_{ss}, V_{dd})$ 
10:   $graph_l \leftarrow GlobalRoute(graph_l, Rule, Parameters)$ 
11:  if  $graph_l$  has crossings or excessive capacity then
12:    goto line 6
13:  end if
14:   $graph_l \leftarrow DetailedRoute(graph_l, Rule)$ 
15:  if  $graph_l$  doesn't satisfy the design rule  $Rule$  then
16:    goto line 6
17:  end if
18:   $S \leftarrow S - S_l$ 
19:   $l \leftarrow l + 1$ 
20:   $Result \leftarrow Result \cup graph_l$ 
21: end while
22: Return  $Result$ 

```

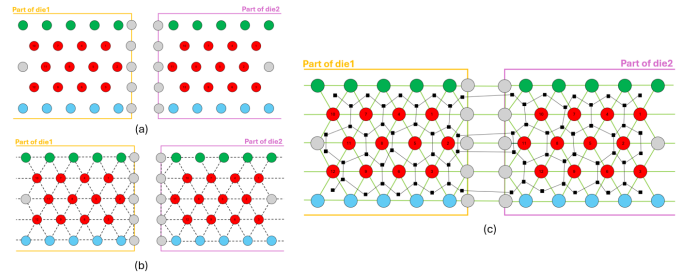


Fig. 7. 以德勞尼三角化建立繞線圖的過程(a)在原本的Die1與Die2的邊界與相鄰距離為 r 的空白處插入Padding bumps(灰色bumps)(b)對原本Bumps與Padding bumps的位置進行三角化，得到三角塊的分割圖(c)根據三角分割圖建立Bump nodes與Tile nodes的連接關係，並將Die1與Die2對應列的Tile nodes進行合併，得出的繞線圖(黑方塊為Tile node，黑塊與黑塊間的黑色連接為Tile-to-tile edges，Bump與Bump之間的淺綠色連接為Bump-to-bump edges，Bump-to-tile edges並無畫出)

進行連接(合併)，形成一個完整的繞線圖，而整個三角化建立繞線圖的過程如Fig. 7。

B. Genetic Algorithm

接著我們會基於基因演算法在繞線圖上對當層要繞線的Signal pairs進行Global route，而當層沒有要繞線的Signal bumps視作Via並向左延伸出Offset via，那些要繞線的Signal bumps以集合 S ，每一組Signal pair在在Die1與Die2的Signal bump為 $s_i^1, s_i^2 \in S$

1) *Representation:* 在表示法上，每個族群 P 中的染色體 $p \in P$ 都是由多個三元數組所組成，三元數組的數量 $|p|$ 為當前層要繞的Signal pairs的數量，可以寫成 $p = (p_1, p_2, \dots, p_{|p|})$ ，而每一個三元數組 $p_i \in p$ 代表一

Algorithm 2 Calculate Number of Crossing

```

1: Input: Integer vectors  $p'_i, q'_i$ , transformed from  $p_i, q_i$ ,
   where  $q \neq p$ , and  $p, q$  use the same channel
2: Output: Number of crossings  $n$ 
3:  $n \leftarrow 0$ 
4:  $f \leftarrow 0$ 
5: for index  $k$  from 1 to  $|p'_i|$  do
6:   if  $p'_{i,k} = 0$  or  $q'_{i,k} = 0$  then
7:     Do nothing
8:   else if  $p'_{i,k} > q'_{i,k}$  then
9:     if  $f = -1$  then
10:       $n \leftarrow n + 1$ 
11:     end if
12:      $f \leftarrow 1$ 
13:   else if  $p'_{i,k} < q'_{i,k}$  then
14:     if  $f = 1$  then
15:       $n \leftarrow n + 1$ 
16:     end if
17:      $f \leftarrow -1$ 
18:   end if
19: end for
20: Return  $n$ 

```

在繞線結果上就會產生交錯問題，因此會貢獻一個交錯數，並更新上下關係，而唯有的這兩線路的編碼之間，為有一方在兩者不為0的位置都不比對方小，此時才不會有交錯情況，這個計算方法可以非常優雅的算出線路的交錯關係，以此方法對任意兩條相同通道的但不同Signal pair的繞線進行比較並加總作為Conflict count。

2) *Excessive Capacity*: 對於每條三角邊，可以根據其兩端的Bumps進行分類，原本每條三角邊的長度為 r ，若其相鄰的Bumps若不是Padding bump則長度要縮減 $V_p + L_b$ (意即相鄰一個非Padding bump則邊的長度剩 $r - V_p - L_b$ ，相鄰兩個非Padding bump則邊的長度剩 $r - 2V_p - 2L_b$ ，因為Padding bump僅是作為填充用途且不具實體)，若該三角邊有Offset via經過(水平邊)，則長度再縮減 $V_o + V_p$ ，最後算出的容量會考慮Ground shielding與剩餘的長度 d ，將該三角邊的capacity設為 $\frac{d/(L_b+L_w)-1}{2}$ ，而Excessive capacity的算法會將染色體的每個三元數組轉換成實際的電路繞線，並更新每一個三角邊所通過的線數，而那些三角邊所超出的線數加總就是Excessive capacity，而唯有當電路的Excessive capacity為0時，後續的Detailed route才有機會繞出滿足設計規則的結果。

3) *Estimated Wire Length*: 在Global route的階段，繞線結果只能反映線路的大致走向，因此只能設計一個估計的值來反映Detailed route可能的趨勢，因此在計算上，我們以三角格的中心點作為繞線的估計位置，並以中心點之間的歐式距離作為估計值，Estimated wire length的算法是將染色體中的每個三元數組所表示的繞線結果以上述方式計算估計值並加總起來。

D. Ground Shielding

當所有Signal nets繞完以後會先決定Signal nets之間的上下關係，之後再沿著每條Signal net的上下方跟著長出相同走法的Groung net(意即相同的三元數組，只是繞的

是Groung net)，之後會交給Detailed Route將這些線路一起繞出，並將Groung net重疊部分進行合併。

E. Detailed Route

在本階段，我們會根據Global route的繞線規劃將決定金屬線的實際位置，在此階段仍然是採用原本的繞線圖進行繞線，繞線方法會根據Global route所決定的上下關係，以由上而下的順序進行繞線。在該演算法中，每一個三角邊可以根據其水平情況分成底邊與斜邊，底邊是允許垂直繞線經過的邊，反之則為斜邊，每個三角邊都會分配其容量數個位置平均分配至整個可繞線區段(不會太靠近Bumps的區段)，每條線只能由這些位置進行繞線，且每個位置都最多只能由同一種線佔據。方法是根據由上至下的順序對每條線進行繞線，每條線都會去走訪Global route所規劃的三角格，並根據現在與下一步三角邊的種類與方向選擇下一步的位置，選擇規則如下，Fig 9.為該規則的所有情況與偏好位置:

- 1) 現在斜邊，下一步底邊，往右上走，越左邊越好
- 2) 現在斜邊，下一步底邊，往右下走，越右越好
- 3) 現在斜邊，下一步斜邊，往右走，越上面越好
- 4) 現在底邊，下一步斜邊，往右上走，越上面越好
- 5) 現在底邊，下一步斜邊，往右下走，越上面越好

當現在繞的是Ground net且上一個被占據的偏好位置也是Ground net時，可以讓這兩條Ground net共用此位置，以此將夾在Signal nets之間的Ground nets進行合併。以上演算法是基於越上方的線要往越上方(或外圍)繞的直覺來實現，在所有三角格都是正三角形且容量足夠的情況下，此方法是繞的出可行設計的，但因為實際三角邊的長度會因為Padding bumps、Offset vias等影響，導致邊長不同，因此在水平方向所均勻分配的位置並不等高，所以水平線路會出現無意義的彎曲，在垂直方向上會因為斜邊與底邊的夾角過大，導致上下移動的線路擠在一起，而不滿足設計規則，因此本方法只是做為本期末專案的暫時解，未來我希望嘗試基於Simulated annealing的方式搭配本方法實現可調整的Detailed route，但目前我先以此方法展示本專案的結果。

V. RESULTS

在本章節，我們進行了四項實驗來比較基於基因演算法與基於最短路徑演算法所實現的Global route結果，並展示基因演算法在本問題上的收斂曲線以及時間複雜度與特殊成本函數上的表現，我們製作了以下5種測資:

- 1) Standard_44: 有44個Signal pairs的且有較差固定順序的標準測資
- 2) Standard_100: Signal pairs較多且通道較大的標準測資
- 3) Unordered_128: 有100個亂序Signal pairs且通道容量較大的測資
- 4) Perfect_100: 有100個最佳順序的Signal pairs且通道容量非常大的測資
- 5) Holed_100: 有100個亂序Signal pairs，且電路中間會有非常多空隙的測資

而我們基因演算法的參數表如下表，而每個實驗都會進行60次並取平均，來減少隨機性帶來的偏誤。

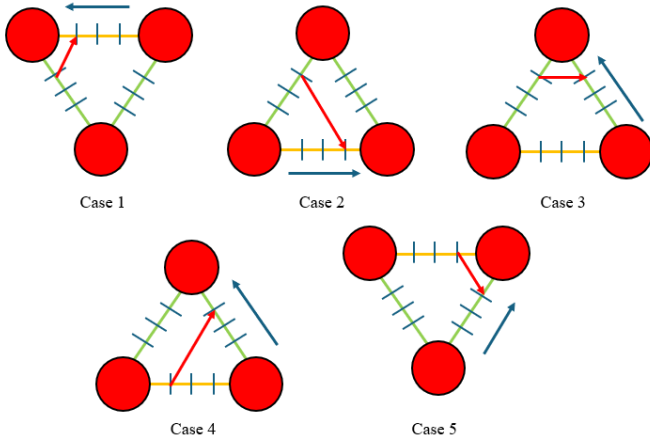


Fig. 9. Detailed route演算法的選擇策略表示圖，綠色邊代表斜邊，黃色邊代表底邊，中間藍色均勻分佈的標示代表可繞的位置，紅色箭頭代表線路方向，藍色箭頭代表該三角邊的位置傾向，每個Case都對應原本的選擇規則。

TABLE I
GENETIC ALGORITHM PARAMETERS

Item	Description
Representation	A set of ternary strings
Crossover	Two-point crossover for each ternary string
Mutation	Random segment setting
Crossover probability (p_c)	90%
Mutation probability (p_m)	100%
Initialization	Assign a random value to all elements in a ternary string (only in effective segment)
Population size (μ)	100
Generations	200
Parent selection	Rank-based roulette wheel selection
Survivor selection	$\mu + \lambda$ with 50% of random selection

A. Comparison

首先要比較我們的方法與基於最短路徑演算法實現的Global route的結果，比較的標的為整個電路中所有Signal pairs的估計路線的加總以及繞完整個電路所需的層數，最短路徑演算法在走訪時，每一個三角格所考慮的成本函數如下：

$$Cost(g_k) = \alpha \times Cost_1(g_k) + Cost_2(g_k) + \beta \times CapacityUtilization(g_k)$$

g_k 是通道中的某個三角格， $Cost(g_k)$ 代表在由起點到此三角格的成本函數，這裡的 $Cost_1$ 與 $Cost_2$ 與基因演算法的部分相同，只是只考慮由起點至 g_k 的路徑，而 $CapacityUtilization$ 是整個電路加上該路徑所經過的每個三角邊的容量使用率，當通過的邊數越接近可用容量則趨近為1，當超過時則固定為1(因為此時 $Cost_1(g_k)$ 中的Excessive capacity會遠大於使用率帶來的影響，因此該數值在超過容量下對路徑選擇影響非常低)， β 是調整參數，通常正比於Bumps間的距離 r ，此設計可以有效防止最短路徑演算法只考慮線長，導致的線路過度壅擠而產生Excessive capacity。在變化策略上，每次最短路徑演算法走完後會選隨機數量個最差(成本最高)的路徑拔除並重繞。

	Method	Case 1 Standard 44	Case 2 Standard 100	Case 3 Unordered 128	Case 4 Perfect 100	Case 5 Holed 100
Average Estimated Wire Length	Ours	51737.53	274098.37	304040.05	256435.27	352754.63
	Shortest Path	52118.19	274575.20	306375.37	264617.97	355278.10
Standard Deviation of Estimated Wire Length	Ours	95.07	243.51	351.30	94.77	497.15
	Shortest Path	182.99	313.89	503.61	1003.13	718.02
T Value of Estimated Wire Length		9.54026E-25	1.47101E-15	1.07162E-52	2.98364E-18	9.24437E-42
Average Number of Layers	Ours	5.03	8.6	5	2.00	5
	Shortest Path	5	8.73	6	2.00	5.23
Standard Deviation of Number of Layers	Ours	0.18	0.49	0.00	0.00	0.00
	Shortest Path	0.00	0.45	0.00	0.00	0.43
T Value of Number of Layers		0.16	0.12	X (STD=0)	X (STD=0)	8.04732E-05

Fig. 10. 基因演算法與最短路徑演算法在相同計算成本的次數下的表現

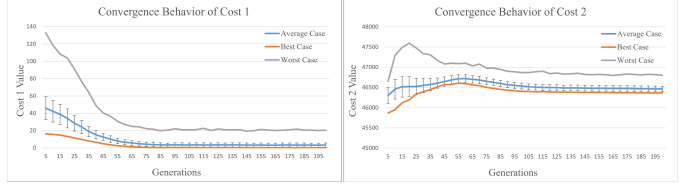


Fig. 11. 基因演算法在 $Cost_1$ 與 $Cost_2$ 的收斂情況，藍色線為族群平均，橙色線與灰色為分別為族群中最好與最差的染色體，黑色垂直線為群體的標準差

實驗選擇每一層所要繞線的Signal pairs的方法是先選Die 1最右邊的Signal bump所對應的Signal bump，每次選的數量是我給的上界(兩方法的上界相同，且是一定在無法在同一層繞出來的上界)，當Global route違反限制後當層繞的數量就會-1，再重新進行Global route，直到沒有違反限制，一層一層繞直到所有Signal pairs都繞完為止。比較上因為最短路徑演算法每走一格後都要去計算附近格的成本函數，所以計算成本函數的頻率會遠高於基因演算法，意即最短路徑演算法的每次變化都是以現有資訊去選擇，但基因演算法的變化是依照過去的資訊趨勢去選擇，因此每一次變化的效益會遠低於最短路徑演算法，所以我們比較的時間若以變化次數來說，最短路徑演算法是遠勝於基因演算法的，但其原因是因為最短路徑演算法的每一步其實都是在作基因演算法的Evaluate，所以我們的比較時間為呼叫成本函數的次數，因此其結果的意義代表在相同資訊量(計算成本的次數)下，我們的演算法與最短路徑演算法的表現的比較。而結果如Fig. 10，可以看到在相同計算成本次數的情況下，除了較小的Standard 44外，其餘較大的測資在層數或線長上平均也僅比最短路徑演算法好了些微，而由各個測資的t分數可以看出我們的方法在此成本函數上與最短路徑演算法的能力並無不同，但至少可以與最短路徑演算法一樣好，因為在之後實際的電路有更多可以考慮的事情。

B. Convergence

為了觀察基因演算法的收斂情況，我以測資Unordered 128進行實驗，並以當層一次繞26組Signal pairs的狀況來觀察收斂情形，結果如Fig. 11，在收斂曲線上可以觀察到代表限制式的 $Cost_1$ 可以正常下降，而當演化次數增加時，群體平均與標準差都有減少趨勢，而標準差在後期仍保有一定值，原因在於我們使用較高的變異率，因為我們在實驗過程觀察到在後期很容易有些微的限制式無法滿足，但因為線段的特性，導致要跳出限制的情況下往往需要較大幅度的變化，因此我們採用有大幅變化的方法來解

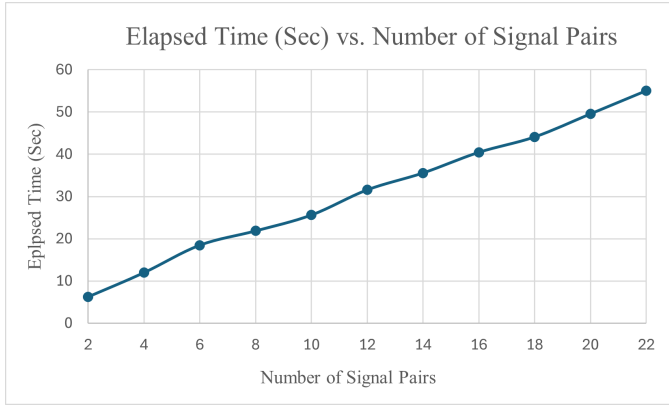


Fig. 12. 繞線時間與的Signal pairs數比較圖

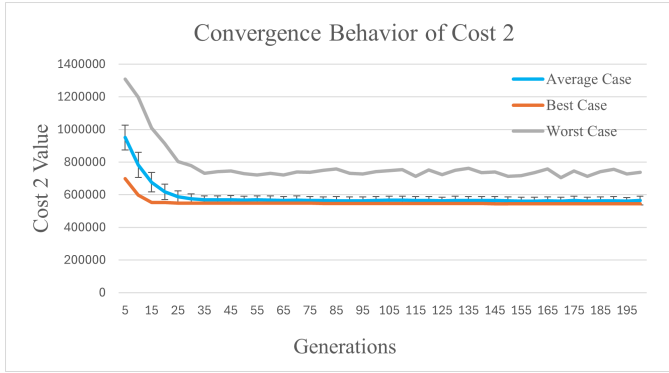


Fig. 13. 加入線長匹配的 $Cost_2$ 收斂曲線，藍色線為族群平均，橙色線與灰色為分別為族群中最好與最差的染色體，黑色垂直直線為群體的標準差

決此問題。而在 $Cost_2$ 的收斂曲線中，可以看到呈現一個先上升後微幅下降的趨勢，這是因為演算法會優先處理限制式的部分，所以會犧牲線長，而當到達峰點處，也代表限制式大多被滿足或無法再下降了，此時才會著重優化線長，但因為會被限制在合法空間上，所以解的下降幅度也較有限，而在兩種 $Cost$ 中，後期都保有足夠的多元性，可以讓我們的電路更容易跳出區域解。

C. Time Complexity

在觀察Signal pairs數與繞線時間的關係上，我選擇Standard 44，從2 Signal pairs到一次22 Signal pair進行繞線，可以看到在繞線的所需時間上，會隨著每次要繞的Signal pairs而上升，而上升的關係約呈現線性的關係，因此我們的方法在程式寫的得宜，其時間表現應該要比最短路徑法來的更有效率。

D. Dynamic Cost Function

雖然在一般成本函數上的表現與最短路徑演算法差不多，但使用基因演算法繞線的優勢在於可以採用動態或較為特殊的成本函數，在這部分的優化能力是最短路徑演算法無法處理的(因為可能出現路徑越長反而成本越小的情況)，像是電路可能有線長匹配等機制，需要透過額外的繞路來減少配對線長的差距，亦或是本問題在實務上會接受相鄰兩組Signal pairs包在同一個Ground shielding，以換取更少的繞線層數，而這些有線與線關係的動態成本

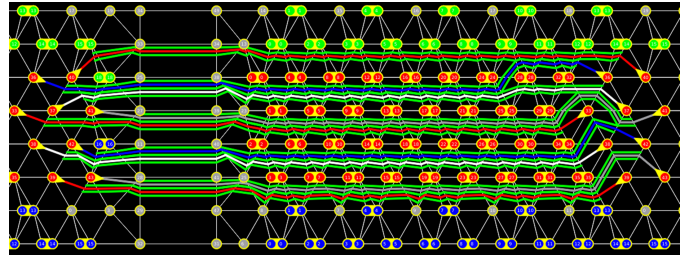


Fig. 14. Standard 44部分繞線結果

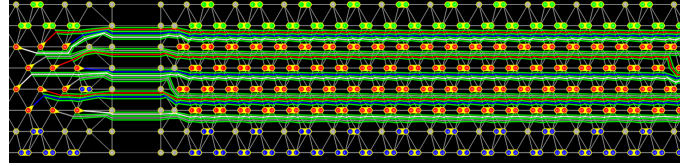


Fig. 15. Standard 100部分繞線結果

就很適合透過基因演算法來優化，而為了驗證我們方法對動態成本函數的優化能力，我們以Unorder 128搭配線長匹配並減少當層要繞的Signal pairs，以著重在 $Cost_2$ 的優化上，而我們以下面函數作為新的 $Cost_2$ ，

$$Cost_2(p) = EstimatedWireLength(p) + \gamma LengthMatching(p)$$

γ 是固定參數，實驗上設為1.1， $LengthMatching(p)$ 是所有有線長匹配的Signal pairs間線長差的絕對值加總，結果如Fig 13，雖然加入了這種有關聯性的成本函數，但基因演算法仍可以優化繞線，並減少線長匹配的成本。

E. Some Cases

剩餘部分則展示一些到目前的方法在所有測資上的情況，Fig 14.至Fig 19.都是基於實驗參數繞出來的結果，綠色，藍色，紅色，及灰色圓點，分別代表VSS bump, VDD bump, Signal bump與Padding bump，綠色線路為Ground shielding，其餘顏色的線路為Signal nets，因為Detailed route的部分並沒有非常完善，所以會有些微折角或擠在一起，但從大致路徑足以看出基因演算法所規劃的Global route是可以優化線長且避免交錯的。

VI. CONCLUSION AND FUTURE WORK

在本篇論文裡，我們提出了一套完整的演算法來實現重佈線層的繞線，並基於三角格繞線圖的架構提出基於基因演算法進行Global route的方法，在此方法中同時考慮到

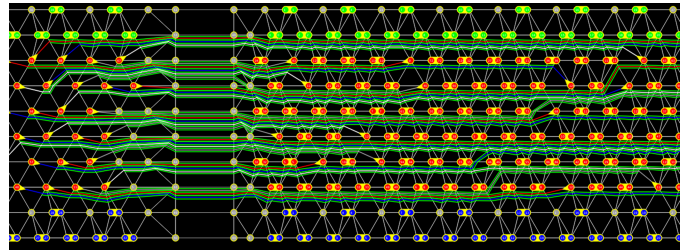


Fig. 16. Unordered 128部分繞線結果

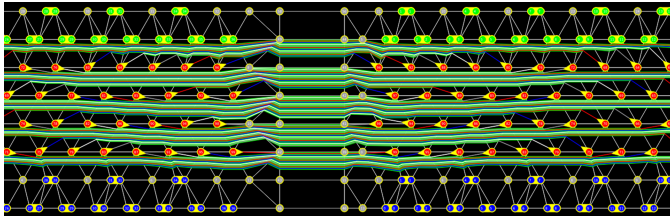


Fig. 17. Perfect 100部分繞線結果

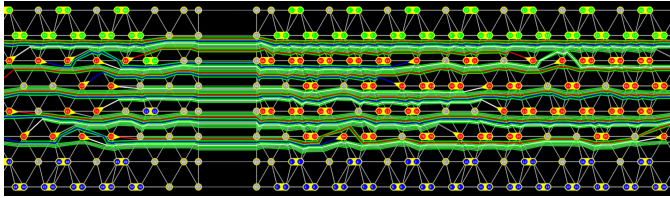


Fig. 18. Holed 100部分繞線結果

線長、邊容量以及交錯等情況，且提出一個非常優雅的演算法，使設計在線路間沒有實際上下關係的Global route階段就可以直接從編碼得出交錯狀態，在Detailed route也提出一個機於貪婪法所實現的方法來決定真正的繞線位置，雖然簡單但堪用，也因此打通了整個Signal pairs的繞線流程，雖然最後的實驗結果顯示了我們的基因演算法在計算相同次數的成本函數下，結果僅比最短路徑法搭配簡單策略好一些，且不具足夠信心水準，但將基於基因演算法在繞線問題上的優勢在於成本函數的設計可以更為自由，可以考慮更具有全局概念的成本函數，而後面的結果也證明我們的方法足以應付這些遠大於真實情況的測資，然而整個演算法在未來仍有許多可以改良的地方，像是基因演算法的策略與參數上仍然是有很大的探索空間，也非常可惜在截稿前我們並沒有找出一個足夠強大的辦法成功超越基於最短路徑演算法的方法，因此這部分是我們希望未來能再挑戰並改善的部分，而最後也感謝這堂課帶給我對本問題與演化計算的啟發還有學術研究上的知識與練習練習，令我受益良多，謝謝教授與課程的助教們。

REFERENCES

- [1] H.-J. Chang, Y.-H. Chen, H.-W. Huang, Y. Yeh, H.-M. Chen, and C.-N. J. Liu, "On Awareness of Offset-Via and Teardrop in Advanced

Packaging Interconnect Synthesis," in Proceedings of the 30th Asia and South Pacific Design Automation Conference, New York, NY, USA: Association for Computing Machinery, 2025, pp. 774–780.

- [2] J. Lienig and K. Thulasiraman, "A Genetic Algorithm for Channel Routing in VLSI Circuits," in Evolutionary Computation, vol. 1, no. 4, pp. 293-311, Dec. 1993, doi: 10.1162/evco.1993.1.4.293.
- [3] Y. -T. Chen and Y. -W. Chang, "Obstacle-Avoiding Multiple Redistribution Layer Routing with Irregular Structures*," 2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD), San Diego, CA, USA, 2022, pp. 1-6.
- [4] Y. Li, D. Dong and X. Guo, "Mobile Robot Path Planning based on Improved Genetic Algorithm With A-star Heuristic Method," 2020 IEEE 9th Joint International Information Technology and Artificial Intelligence Conference (ITAIC), Chongqing, China, 2020, pp. 1306-1311, doi: 10.1109/ITAIC49862.2020.9338968. keywords: Heuristic algorithms;Simulation;Turning;Stability analysis;Trajectory;Mobile robots;Genetic algorithms;A-Star;Genetic Algorithm(GA);Mobile Robot;Path planning,

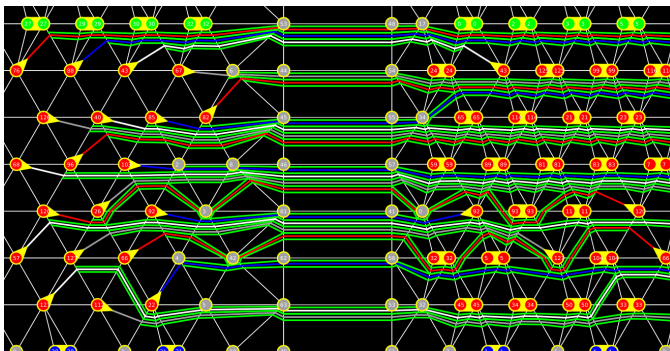


Fig. 19. Unordered 128加上線長匹配的部分繞線結果，Signal pairs 120與66為了匹配其他較長的線路，故意出現繞路以減少差距